

LAPORAN TUGAS 7

Main Memory



Dosen Pengampu Mata Kuliah:

Triawan Adi Cahyanto, M.Kom

Disusun Oleh:

Mohamad Qowi Aizani (2410651003)

Kelas A

18 Desember 2025

**PRODI TEKNIK INFORMATIKA
FAKULTAS TEKNIK
UNIVERSITAS MUHAMMADIYAH
JEMBER 2025**

BAB I

PENDAHULUAN

1.1 Latar Belakang

Perkembangan sistem operasi modern menuntut pengelolaan memori yang efisien agar proses dapat berjalan secara optimal, aman, dan stabil. Memori utama (main memory) merupakan komponen penting yang berfungsi sebagai tempat penyimpanan data dan instruksi yang sedang dieksekusi oleh CPU. Keterbatasan kapasitas memori fisik menyebabkan sistem operasi harus memiliki mekanisme manajemen memori yang baik, seperti alokasi memori, paging, page table, dan swapping.

Pada praktikum Main Memory ini, mahasiswa mempelajari konsep dasar manajemen memori secara teori dan praktik melalui simulasi menggunakan bahasa pemrograman C pada sistem operasi Linux. Praktikum ini mencakup alokasi memori berurutan (contiguous allocation), mekanisme paging, struktur page table satu level dan multi-level, swapping, serta analisis arsitektur memori pada prosesor Intel dan ARM. Dengan adanya praktikum ini, diharapkan mahasiswa dapat memahami bagaimana sistem operasi mengelola memori secara nyata, bukan hanya secara konseptual.

1.2 Tujuan Praktikum

Tujuan dari pelaksanaan Praktikum 7: Main Memory adalah sebagai berikut:

1. Memahami konsep dasar manajemen memori dalam sistem operasi.
2. Mengimplementasikan dan menganalisis algoritma alokasi memori berurutan (First-Fit, Best-Fit, dan Worst-Fit).
3. Memahami mekanisme paging serta proses translasi alamat logis ke alamat fisik.
4. Mengimplementasikan dan menganalisis struktur page table, termasuk two-level page table.
5. Memahami konsep swapping serta membandingkan strategi FIFO, LRU, dan Priority-based.

6. Menganalisis perbedaan arsitektur manajemen memori pada prosesor Intel dan ARM.

1.3 Spesifikasi Sistem

Spesifikasi sistem yang digunakan dalam pelaksanaan praktikum ini adalah sebagai berikut:

- Sistem Operasi : Linux Ubuntu
- Bahasa Pemrograman : C
- Compiler : GCC (GNU Compiler Collection)
- Editor Teks : Nano / Vim / Gedit
- Tools Pendukung : GDB, Valgrind (opsional)
- Arsitektur Sistem : x86 / x86-64 (Intel) dan simulasi ARM
- Lingkungan Praktikum : Terminal Linux

Spesifikasi sistem tersebut digunakan untuk memastikan seluruh program dan simulasi manajemen memori dapat berjalan dengan baik sesuai dengan skenario praktikum yang telah disediakan.

BAB II

DASAR TEORI

2.1 Memory Management

Manajemen memori adalah salah satu fungsi utama sistem operasi yang bertanggung jawab mengatur penggunaan memori utama agar dapat digunakan secara efisien oleh berbagai proses yang berjalan secara bersamaan. Tugas utama manajemen memori meliputi pengalokasian memori untuk proses, translasi alamat logis ke alamat fisik, perlindungan memori antar proses, serta optimasi penggunaan memori agar tidak terjadi pemborosan. Dengan manajemen memori yang baik, sistem operasi mampu menjalankan banyak proses secara bersamaan tanpa saling mengganggu.

2.2 Contiguous Memory Allocation

Contiguous memory allocation adalah metode pengalokasian memori di mana setiap proses ditempatkan pada satu blok memori yang berurutan (kontigu). Metode ini mudah diimplementasikan dan memiliki waktu akses yang cepat, namun memiliki kelemahan berupa external fragmentation. Untuk mengatasi hal tersebut, digunakan beberapa algoritma alokasi seperti First-Fit, Best-Fit, dan Worst-Fit. First-Fit memilih blok kosong pertama yang cukup besar, Best-Fit memilih blok terkecil yang masih mencukupi, sedangkan Worst-Fit memilih blok terbesar yang tersedia.

2.3 Paging

Paging merupakan teknik manajemen memori yang membagi memori fisik menjadi blok berukuran tetap yang disebut frame, dan memori logis menjadi page dengan ukuran yang sama. Dengan paging, proses tidak perlu ditempatkan secara berurutan dalam memori fisik sehingga dapat menghilangkan external fragmentation. Sistem operasi menggunakan mekanisme translasi alamat untuk mengubah alamat logis menjadi alamat fisik melalui bantuan page table.

2.4 Page Table Structures

Page table adalah struktur data yang digunakan untuk menyimpan pemetaan antara page pada memori logis dengan frame pada memori fisik. Terdapat beberapa struktur page table, antara lain single-level page table, multi-level page table (hierarchical paging),

hashed page table, dan inverted page table. Multi-level page table digunakan untuk mengurangi penggunaan memori dengan hanya mengalokasikan tabel yang dibutuhkan, terutama pada sistem dengan ruang alamat virtual yang besar.

2.5 Swapping

Swapping adalah mekanisme pemindahan proses dari memori utama ke media penyimpanan sekunder (disk) ketika memori utama tidak mencukupi. Proses yang dipindahkan ke disk disebut swap out, sedangkan proses yang dimasukkan kembali ke memori utama disebut swap in. Swapping memungkinkan sistem menjalankan lebih banyak proses dibandingkan kapasitas memori fisik, namun memiliki overhead waktu yang cukup besar karena melibatkan operasi I/O ke disk.

2.6 Intel 32 & 64-bit Architectures

Arsitektur Intel 32-bit (IA-32) memiliki ruang alamat virtual sebesar 4 GB dengan mekanisme paging dua level. Sementara itu, arsitektur Intel 64-bit (x86-64) mendukung ruang alamat virtual yang jauh lebih besar, hingga 256 TB secara praktis, dengan mekanisme paging empat level. Perbedaan ini memungkinkan sistem 64-bit mengelola memori dalam jumlah besar secara lebih efisien dibandingkan sistem 32-bit.

2.7 ARMv8 Architectures

ARMv8 merupakan arsitektur prosesor modern yang mendukung mode 32-bit (AArch32) dan 64-bit (AArch64). ARMv8 menggunakan mekanisme paging multi-level dengan ukuran page yang fleksibel, seperti 4 KB, 16 KB, atau 64 KB. Arsitektur ini dirancang untuk efisiensi daya dan performa tinggi, sehingga banyak digunakan pada perangkat mobile, embedded system, serta server modern.

BAB III

Jawaban Tugas Praktikum

3.1.1 Analisis Memory Layout

Tugas ini bertujuan untuk memahami dan menganalisis layout memori proses pada sistem operasi Linux melalui modifikasi program `address_demo.c`. Mahasiswa diminta menambahkan beberapa fitur pada program untuk menampilkan alamat memori dari berbagai jenis data dan fungsi, sehingga dapat diamati bagaimana sistem operasi mengatur ruang alamat suatu proses.

Dalam tugas ini, dilakukan analisis terhadap beberapa segment memori utama, yaitu text segment, data segment, heap, dan stack. Program dimodifikasi untuk menampilkan alamat array statis dan dinamis, alamat beberapa fungsi yang berbeda, alamat variabel lokal pada pemanggilan fungsi rekursif, serta alamat string literal. Dengan menampilkan alamat-alamat tersebut, mahasiswa dapat mengamati perbedaan lokasi dan karakteristik setiap segment memori.

Selain itu, melalui penggunaan fungsi rekursif, tugas ini juga menunjukkan bagaimana stack bekerja dan bagaimana arah pertumbuhan stack saat terjadi pemanggilan fungsi secara berulang. Penggunaan alokasi memori dinamis dengan `malloc()` memperlihatkan perilaku heap yang bertumbuh ke arah alamat memori yang lebih besar.

Melalui tugas ini, mahasiswa diharapkan mampu memahami konsep memory layout, arah pertumbuhan heap dan stack, serta pengaruh mekanisme Address Space Layout Randomization (ASLR) terhadap perubahan alamat memori setiap kali program dijalankan, tanpa mengubah struktur segment memori itu sendiri.

3.1.2 Source Code

```
#include <stdio.h>
#include <stdlib.h>

/* Global variables */
int global_var = 100;
```

```

static int static_array[5] = {1,2,3,4,5};

/* Multiple functions */
void func1() {}
void func2() {}
void func3() {}

/* Recursive function */
void recursive_func(int level, int max) {
    int local_var = level;
    printf("  Level %d local_var address: %p\n", level, (void*)&local_var);

    if (level < max) {
        recursive_func(level + 1, max);
    }
}

int main() {
    printf("=== ENHANCED MEMORY LAYOUT ===\n\n");

    /* 1. Text Segment */
    printf("1. Text Segment:\n");
    printf("  main() address : %p\n", (void*)&main);
    printf("  func1() address: %p\n", (void*)&func1);
    printf("  func2() address: %p\n", (void*)&func2);
    printf("  func3() address: %p\n\n", (void*)&func3);

    /* 2. Data Segment */
    char *string_literal = "Hello Memory Layout";
    printf("2. Data Segment:\n");
    printf("  static_array[0] address : %p\n", (void*)&static_array[0]);
    printf("  string_literal address : %p\n\n", (void*)string_literal);

    /* 3. Heap */
    printf("3. Heap:\n");
    int *dynamic_array = (int*)malloc(5 * sizeof(int));
    int *block1 = (int*)malloc(sizeof(int));
    int *block2 = (int*)malloc(sizeof(int));

    printf("  dynamic_array[0] address: %p\n", (void*)&dynamic_array[0]);

```

```

printf("  malloc block 1 address : %p\n", (void*)block1);
printf("  malloc block 2 address : %p\n\n", (void*)block2);

/* 4. Stack (Recursive Calls) */
printf("4. Stack (Recursive Calls):\n");
recursive_func(0, 2);

free(dynamic_array);
free(block1);
free(block2);

return 0;
}

```

3.1.3 Screenshot Out

```

qowi@QOWIAIZANI:~$ nano address_demo.c
qowi@QOWIAIZANI:~$ gcc address_demo.c -o address_demo
qowi@QOWIAIZANI:~$ ./address_demo
=== ENHANCED MEMORY LAYOUT ===

1. Text Segment:
   main() address : 0x5c56720b725c
   func1() address: 0x5c56720b71c9
   func2() address: 0x5c56720b71d4
   func3() address: 0x5c56720b71df

2. Data Segment:
   static_array[0] address : 0x5c56720ba020
   string_literal address  : 0x5c56720b80c2

3. Heap:
   dynamic_array[0] address: 0x5c56987fe6b0
   malloc block 1 address : 0x5c56987fe6d0
   malloc block 2 address : 0x5c56987fe6f0

4. Stack (Recursive Calls):
   Level 0 local_var address: 0x7fffd41905af4
   Level 1 local_var address: 0x7fffd41905ac4
   Level 2 local_var address: 0x7fffd41905a94
qowi@QOWIAIZANI:~$ |

```


3.1.4 Penjelasan Implementasi

Penjelasan Implementasi Tugas 1: Analisis Memory Layout

Implementasi tugas ini dilakukan dengan memodifikasi program `address_demo.c` untuk menampilkan alamat memori dari berbagai komponen program. Modifikasi dilakukan untuk merepresentasikan setiap segment memori utama yang digunakan oleh sebuah proses pada sistem operasi Linux, yaitu text segment, data segment, heap, dan stack.

1. Implementasi Text Segment

Untuk menganalisis text segment, program menampilkan alamat dari beberapa fungsi, yaitu `main()`, `func1()`, `func2()`, dan `func3()`. Alamat fungsi diperoleh dengan menggunakan operator alamat (`&`) pada nama fungsi dan dicetak menggunakan format `%p`. Hasil yang ditampilkan menunjukkan bahwa seluruh fungsi berada pada area memori yang berdekatan, yang menandakan bahwa fungsi-fungsi tersebut disimpan pada text segment.

2. Implementasi Data Segment

Analisis data segment dilakukan dengan mendeklarasikan array statis (`static_array`) dan string literal. Alamat elemen pertama dari array statis dicetak untuk menunjukkan lokasi data yang telah dialokasikan secara statis. Selain itu, alamat string literal ditampilkan untuk memperlihatkan bahwa string literal disimpan pada area memori khusus yang bersifat read-only dan termasuk dalam data segment.

3. Implementasi Heap

Untuk menganalisis heap, program menggunakan fungsi `malloc()` untuk melakukan alokasi memori dinamis. Program mengalokasikan array dinamis dan beberapa blok memori tambahan, kemudian menampilkan alamat hasil alokasi tersebut. Dari hasil yang diperoleh, dapat diamati bahwa alamat heap berada di atas data segment dan bertambah seiring dengan alokasi memori baru, menunjukkan arah pertumbuhan heap ke alamat yang lebih besar.

4. Implementasi Stack

Analisis stack dilakukan menggunakan fungsi rekursif `recursive_func()`. Pada setiap pemanggilan fungsi, sebuah variabel lokal dideklarasikan dan alamatnya ditampilkan. Dengan membandingkan alamat variabel lokal pada setiap level rekursi, dapat diamati bahwa alamat stack semakin kecil, yang menunjukkan bahwa stack bertumbuh ke arah alamat memori yang lebih rendah.

5. Pembersihan Memori

Setelah penggunaan heap selesai, fungsi `free()` digunakan untuk melepaskan memori yang telah dialokasikan secara dinamis. Langkah ini penting untuk mencegah kebocoran memori (*memory leak*) dan menunjukkan praktik pengelolaan memori yang baik.

3.2.1. Deskripsi Tugas 2: Implementasi Memory Allocator

Tugas 2 bertujuan untuk mengembangkan dan menganalisis sebuah simulator memory allocator menggunakan metode alokasi memori berurutan (*contiguous memory allocation*). Pada tugas ini, program memory allocator yang telah tersedia dikembangkan dengan menambahkan algoritma alokasi baru serta fitur analisis performa untuk membandingkan efektivitas beberapa algoritma alokasi memori.

Mahasiswa diminta untuk mengimplementasikan algoritma Next-Fit, yaitu variasi dari algoritma First-Fit yang memulai pencarian blok kosong dari posisi alokasi terakhir. Implementasi ini bertujuan untuk meningkatkan efisiensi pencarian blok memori dibandingkan dengan memulai pencarian dari awal setiap kali alokasi dilakukan.

Selain itu, tugas ini juga menambahkan fitur perhitungan fragmentasi memori, yang meliputi perhitungan total ruang bebas, ukuran blok kosong terbesar, jumlah lubang memori (*free holes*), serta rasio fragmentasi eksternal. Analisis fragmentasi dilakukan untuk mengevaluasi dampak setiap algoritma alokasi terhadap efisiensi penggunaan memori.

Untuk melengkapi analisis, dilakukan benchmark testing terhadap empat algoritma alokasi memori, yaitu First-Fit, Best-Fit, Worst-Fit, dan Next-Fit. Pengujian dilakukan dengan skenario alokasi dan dealokasi proses yang sama untuk setiap algoritma, kemudian dibandingkan berdasarkan waktu eksekusi dan tingkat fragmentasi yang dihasilkan.

Melalui Tugas 2 ini, mahasiswa diharapkan dapat memahami perbedaan karakteristik setiap algoritma alokasi memori, menganalisis trade-off antara kecepatan alokasi dan fragmentasi memori, serta mengevaluasi algoritma yang paling sesuai untuk kondisi sistem tertentu.

3.2.2 Source Code

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <time.h>

#define MAX_BLOCKS 20
#define MEMORY_SIZE 1000

typedef struct {
    int id;
    int size;
    int start_address;
    int is_allocated;
} MemoryBlock;

MemoryBlock memory[MAX_BLOCKS];
int block_count = 0;

/* === VARIABEL GLOBAL UNTUK NEXT-FIT === */
int last_alloc_index = 0;

/* ===== INISIALISASI ===== */
void initialize_memory() {
    memory[0].id = -1;
    memory[0].size = MEMORY_SIZE;
```

```

    memory[0].start_address = 0;
    memory[0].is_allocated = 0;
    block_count = 1;
    last_alloc_index = 0;
}

/* ===== DISPLAY ===== */
void display_memory() {
    printf("\n=== STATUS MEMORI ===\n");
    printf("%-10s %-10s %-15s %-10s\n",
        "Block", "Size", "Start Addr", "Status");
    printf("-----\n");

    for (int i = 0; i < block_count; i++) {
        printf("%-10d %-10d %-15d %-10s\n",
            memory[i].id,
            memory[i].size,
            memory[i].start_address,
            memory[i].is_allocated ? "Alloc" : "Free");
    }
}

/* ===== FIRST-FIT ===== */
int first_fit(int pid, int size) {
    for (int i = 0; i < block_count; i++) {
        if (!memory[i].is_allocated && memory[i].size >= size) {
            int old_size = memory[i].size;
            int old_start = memory[i].start_address;

            memory[i].id = pid;
            memory[i].size = size;
            memory[i].is_allocated = 1;

            if (old_size > size) {
                for (int j = block_count; j > i + 1; j--)
                    memory[j] = memory[j - 1];

                memory[i + 1] = (MemoryBlock){-1, old_size - size, old_start + size, 0};
                block_count++;
            }
        }
    }
}

```

```

        return 1;
    }
}
return 0;
}

/* ===== BEST-FIT ===== */
int best_fit(int pid, int size) {
    int best = -1, smallest = MEMORY_SIZE + 1;

    for (int i = 0; i < block_count; i++)
        if (!memory[i].is_allocated && memory[i].size >= size &&
            memory[i].size < smallest) {
            smallest = memory[i].size;
            best = i;
        }

    if (best == -1) return 0;

    int old_size = memory[best].size;
    int old_start = memory[best].start_address;

    memory[best] = (MemoryBlock){pid, size, old_start, 1};

    if (old_size > size) {
        for (int j = block_count; j > best + 1; j--)
            memory[j] = memory[j - 1];

        memory[best + 1] = (MemoryBlock){-1, old_size - size, old_start + size, 0};
        block_count++;
    }
    return 1;
}

/* ===== WORST-FIT ===== */
int worst_fit(int pid, int size) {
    int worst = -1, largest = -1;

    for (int i = 0; i < block_count; i++)

```

```

    if (!memory[i].is_allocated && memory[i].size > largest &&
        memory[i].size >= size) {
        largest = memory[i].size;
        worst = i;
    }

    if (worst == -1) return 0;

    int old_size = memory[worst].size;
    int old_start = memory[worst].start_address;

    memory[worst] = (MemoryBlock){pid, size, old_start, 1};

    if (old_size > size) {
        for (int j = block_count; j > worst + 1; j--)
            memory[j] = memory[j - 1];

        memory[worst + 1] = (MemoryBlock){-1, old_size - size, old_start + size, 0};
        block_count++;
    }
    return 1;
}

/* ===== NEXT-FIT ===== */
int next_fit(int pid, int size) {
    int start = last_alloc_index;

    for (int i = 0; i < block_count; i++) {
        int idx = (start + i) % block_count;

        if (!memory[idx].is_allocated && memory[idx].size >= size) {
            int old_size = memory[idx].size;
            int old_start = memory[idx].start_address;

            memory[idx] = (MemoryBlock){pid, size, old_start, 1};
            last_alloc_index = idx;

            if (old_size > size) {
                for (int j = block_count; j > idx + 1; j--)
                    memory[j] = memory[j - 1];
            }
        }
    }
}

```

```

        memory[idx + 1] =
            (MemoryBlock){-1, old_size - size, old_start + size, 0};
        block_count++;
    }
    return 1;
}
}
return 0;
}

/* ===== DEALLOCATE ===== */
void deallocate(int pid) {
    for (int i = 0; i < block_count; i++) {
        if (memory[i].id == pid && memory[i].is_allocated) {
            memory[i].id = -1;
            memory[i].is_allocated = 0;

            if (i + 1 < block_count && !memory[i + 1].is_allocated) {
                memory[i].size += memory[i + 1].size;
                for (int j = i + 1; j < block_count - 1; j++)
                    memory[j] = memory[j + 1];
                block_count--;
            }
            return;
        }
    }
}

/* ===== FRAGMENTATION ===== */
void calculate_fragmentation() {
    int total_free = 0, largest = 0, holes = 0;

    for (int i = 0; i < block_count; i++) {
        if (!memory[i].is_allocated) {
            total_free += memory[i].size;
            holes++;
            if (memory[i].size > largest)
                largest = memory[i].size;
        }
    }
}

```

```

    }

    double ext_frag = total_free ? 1.0 - ((double)largest / total_free) : 0;

    printf("\n--- Fragmentation ---\n");
    printf("Total Free Space : %d\n", total_free);
    printf("Largest Free    : %d\n", largest);
    printf("Free Holes      : %d\n", holes);
    printf("Ext Fragmentation: %.2f\n", ext_frag);
}

/* ===== BENCHMARK ===== */
void benchmark_test(int algo) {
    clock_t start, end;
    initialize_memory();

    start = clock();

    int (*alloc)(int, int);
    if (algo == 1) alloc = first_fit;
    else if (algo == 2) alloc = best_fit;
    else if (algo == 3) alloc = worst_fit;
    else alloc = next_fit;

    alloc(1,100); alloc(2,200); alloc(3,300); alloc(4,250); alloc(5,150);
    deallocate(2); deallocate(4);
    alloc(6,180); alloc(7,220); alloc(8,100);

    end = clock();

    display_memory();
    calculate_fragmentation();

    printf("Execution Time: %f sec\n",
        (double)(end - start) / CLOCKS_PER_SEC);
}

/* ===== MAIN ===== */
int main() {
    int choice;

```



```

while (1) {
    printf("\n=== CONTIGUOUS MEMORY ALLOCATION ===\n");
    printf("1. First-Fit\n2. Best-Fit\n3. Worst-Fit\n4. Next-Fit\n");
    printf("5. Benchmark All\n6. Exit\nChoice: ");
    scanf("%d", &choice);

    if (choice == 6) break;

    if (choice == 5) {
        for (int i = 1; i <= 4; i++)
            benchmark_test(i);
    } else if (choice >= 1 && choice <= 4) {
        benchmark_test(choice);
    }
}
return 0;
}

```

3.2.3 Screenshot Output

```

qowi@QOWIAIZANI:~$ ./memory_allocator

=== CONTIGUOUS MEMORY ALLOCATION ===
1. First-Fit
2. Best-Fit
3. Worst-Fit
4. Next-Fit
5. Benchmark All
6. Exit
Choice: 5

=== STATUS MEMORI ===
Block      Size      Start Addr      Status
-----
1           100         0                Alloc
6           180        100              Alloc
-1          20         280              Free
3           300        300              Alloc
7           220        600              Alloc
-1          30         820              Free
5           150        850              Alloc

--- Fragmentation ---
Total Free Space : 50
Largest Free      : 30
Free Holes        : 2
Ext Fragmentation: 0.40
Execution Time: 0.000002 sec

=== STATUS MEMORI ===
Block      Size      Start Addr      Status
-----
1           100         0                Alloc
6           180        100              Alloc
-1          20         280              Free
3           300        300              Alloc
7           220        600              Alloc
-1          30         820              Free
5           150        850              Alloc

```

```

--- Fragmentation ---
Total Free Space : 50
Largest Free     : 30
Free Holes       : 2
Ext Fragmentation: 0.40
Execution Time: 0.000002 sec

=== STATUS MEMORI ===
Block      Size      Start Addr      Status
-----
1          100        0              Alloc
8          100       100              Alloc
-1         100       200              Free
3          300       300              Alloc
6          180       600              Alloc
-1         70        780              Free
5          150       850              Alloc

--- Fragmentation ---
Total Free Space : 170
Largest Free     : 100
Free Holes       : 2
Ext Fragmentation: 0.41
Execution Time: 0.000001 sec

=== STATUS MEMORI ===
Block      Size      Start Addr      Status
-----
1          100        0              Alloc
6          180       100              Alloc
-1         20        280              Free
3          300       300              Alloc
7          220       600              Alloc
-1         30        820              Free
5          150       850              Alloc

--- Fragmentation ---
Total Free Space : 50
Largest Free     : 30
Free Holes       : 2
Ext Fragmentation: 0.40
Execution Time: 0.000002 sec

=== CONTIGUOUS MEMORY ALLOCATION ===
1. First-Fit
2. Best-Fit
3. Worst-Fit
4. Next-Fit
5. Benchmark All
6. Exit
Choice: |

```

3.2.4 Penjelasan Implementasi Tugas 2: Implementasi Memory Allocator

Implementasi Tugas 2 dilakukan dengan mengembangkan program simulasi contiguous memory allocation yang telah disediakan. Program ini menggunakan struktur data MemoryBlock untuk merepresentasikan blok-blok memori yang tersedia dan teralokasi. Setiap blok memori memiliki informasi ukuran, alamat awal, status alokasi, serta identitas proses yang menggunakan blok tersebut.

1. Implementasi Algoritma Next-Fit

Algoritma Next-Fit diimplementasikan sebagai pengembangan dari algoritma First-Fit. Perbedaan utama terletak pada titik awal pencarian blok kosong. Pada Next-Fit, pencarian tidak dimulai dari awal memori, melainkan dari posisi alokasi terakhir yang disimpan dalam variabel global `last_alloc_index`.

Pada setiap permintaan alokasi, algoritma akan melakukan pencarian secara melingkar (*circular search*) hingga menemukan blok kosong yang ukurannya mencukupi. Jika blok yang ditemukan lebih besar dari ukuran yang dibutuhkan, maka blok tersebut akan dibagi menjadi dua bagian, yaitu blok teralokasi dan blok kosong baru. Posisi indeks terakhir akan diperbarui untuk digunakan pada alokasi berikutnya.

2. Implementasi Penggabungan Blok (Coalescing)

Saat proses didealokasi, blok memori yang dibebaskan akan diperiksa terhadap blok di sebelah kiri dan kanan. Jika blok tetangga berstatus bebas, maka dilakukan penggabungan (*coalescing*) untuk membentuk satu blok kosong yang lebih besar. Proses ini bertujuan untuk mengurangi fragmentasi eksternal pada memori.

3. Implementasi Perhitungan Fragmentasi

Untuk menganalisis efisiensi penggunaan memori, ditambahkan fungsi perhitungan fragmentasi. Fungsi ini menghitung total ruang memori bebas, ukuran blok bebas terbesar, serta jumlah lubang memori. Berdasarkan nilai tersebut, dihitung rasio fragmentasi eksternal yang menunjukkan seberapa besar ruang memori bebas yang tidak dapat dimanfaatkan secara optimal.

4. Implementasi Benchmark Testing

Benchmark testing dilakukan untuk membandingkan performa empat algoritma alokasi memori, yaitu First-Fit, Best-Fit, Worst-Fit, dan Next-Fit. Setiap algoritma diuji menggunakan skenario alokasi dan dealokasi proses yang sama. Waktu eksekusi diukur menggunakan fungsi `clock()` untuk mengetahui efisiensi masing-masing algoritma. Hasil benchmark digunakan untuk mengevaluasi trade-off antara kecepatan alokasi dan tingkat fragmentasi yang dihasilkan.

5. Integrasi dengan Menu Program

Seluruh algoritma dan fitur tambahan diintegrasikan ke dalam menu utama program. Pengguna dapat memilih algoritma tertentu atau menjalankan benchmark untuk seluruh algoritma secara berurutan. Hal ini memudahkan proses pengujian dan perbandingan hasil secara langsung.

3.3.1 Deskripsi Tugas 3: Advanced Paging System

Tugas 3 bertujuan untuk mengembangkan sistem paging yang lebih lanjut dengan mengimplementasikan algoritma page replacement serta melakukan analisis kinerja berdasarkan statistik penggunaan memori. Pada tugas ini, program simulator paging yang telah tersedia dimodifikasi agar mampu menangani kondisi ketika seluruh frame di memori fisik telah terisi penuh.

Mahasiswa diminta untuk mengimplementasikan dua algoritma page replacement, yaitu FIFO (First-In First-Out) dan LRU (Least Recently Used). Algoritma FIFO mengganti halaman yang pertama kali dimasukkan ke memori, sedangkan algoritma LRU mengganti halaman yang paling lama tidak diakses. Implementasi kedua algoritma ini bertujuan untuk memahami perbedaan strategi penggantian halaman dan dampaknya terhadap performa sistem.

Selain itu, ditambahkan fitur pencatatan statistik yang meliputi jumlah akses halaman, jumlah page fault, jumlah page hit, nilai hit ratio, serta jumlah disk write yang terjadi akibat penggantian halaman yang memiliki *dirty bit*. Statistik ini digunakan untuk mengevaluasi efisiensi masing-masing algoritma page replacement.

Pada bagian pengujian, sistem diuji menggunakan sebuah reference string dengan jumlah frame tertentu. Hasil pengujian FIFO dan LRU kemudian dibandingkan berdasarkan jumlah page fault, hit ratio, dan disk write. Melalui Tugas 3 ini, mahasiswa diharapkan dapat memahami mekanisme kerja sistem paging secara menyeluruh serta mampu

menganalisis kelebihan dan kekurangan dari setiap algoritma page replacement yang digunakan.

3.3.2 Source Code

```
#include <stdio.h>
#include <stdlib.h>

#define FRAME_SIZE 256
#define NUM_FRAMES 4
#define PAGE_SIZE FRAME_SIZE
#define MAX_PAGES 100

typedef struct {
    int page_number;
    int frame_number;
    int valid;
    int reference_bit;
    int modify_bit;
    int last_used; // untuk LRU
} PageTableEntry;

typedef struct {
    int occupied;
    int page_number;
} Frame;

typedef struct {
    int total_accesses;
    int page_faults;
    int page_hits;
    int disk_writes;
    float hit_ratio;
} PageStatistics;

Frame physical_memory[NUM_FRAMES];
PageTableEntry page_table[MAX_PAGES];
PageStatistics stats;
```

```

int fifo_pointer = 0;
int time_counter = 0;

/* ===== INIT ===== */
void initialize_system() {
    for (int i = 0; i < NUM_FRAMES; i++) {
        physical_memory[i].occupied = 0;
        physical_memory[i].page_number = -1;
    }

    for (int i = 0; i < MAX_PAGES; i++) {
        page_table[i].valid = 0;
        page_table[i].frame_number = -1;
        page_table[i].modify_bit = 0;
        page_table[i].last_used = 0;
    }

    stats.total_accesses = 0;
    stats.page_faults = 0;
    stats.page_hits = 0;
    stats.disk_writes = 0;

    fifo_pointer = 0;
    time_counter = 0;

    printf("Sistem paging diinisialisasi\n");
}

/* ===== FIFO ===== */
int fifo_replace() {
    int frame = fifo_pointer;
    fifo_pointer = (fifo_pointer + 1) % NUM_FRAMES;
    return frame;
}

/* ===== LRU ===== */
int lru_replace() {
    int min = page_table[physical_memory[0].page_number].last_used;
    int frame = 0;

```

```

    for (int i = 1; i < NUM_FRAMES; i++) {
        int page = physical_memory[i].page_number;
        if (page_table[page].last_used < min) {
            min = page_table[page].last_used;
            frame = i;
        }
    }
    return frame;
}

/* ===== PAGE ALLOCATION ===== */
void allocate_page(int page_num, int algorithm) {
    stats.page_faults++;

    int frame = -1;

    // cari frame kosong
    for (int i = 0; i < NUM_FRAMES; i++) {
        if (!physical_memory[i].occupied) {
            frame = i;
            break;
        }
    }

    // jika penuh → page replacement
    if (frame == -1) {
        frame = (algorithm == 1) ? fifo_replace() : lru_replace();
        int old_page = physical_memory[frame].page_number;

        if (page_table[old_page].modify_bit) {
            stats.disk_writes++;
        }

        page_table[old_page].valid = 0;
    }

    physical_memory[frame].occupied = 1;
    physical_memory[frame].page_number = page_num;
}

```

```

    page_table[page_num].frame_number = frame;
    page_table[page_num].valid = 1;
    page_table[page_num].last_used = ++time_counter;

    printf("Page %d dimuat ke frame %d\n", page_num, frame);
}

/* ===== MEMORY ACCESS ===== */
void access_memory(int logical_address, int write, int algorithm) {
    stats.total_accesses++;

    int page = logical_address / PAGE_SIZE;

    if (page_table[page].valid) {
        stats.page_hits++;
    } else {
        printf("PAGE FAULT pada page %d\n", page);
        allocate_page(page, algorithm);
    }

    page_table[page].last_used = ++time_counter;

    if (write) {
        page_table[page].modify_bit = 1;
    }
}

/* ===== STATISTICS ===== */
void display_statistics(char *algo) {
    stats.hit_ratio = (float)stats.page_hits / stats.total_accesses;

    printf("\n=== STATISTIK %s ===\n", algo);
    printf("Total Access : %d\n", stats.total_accesses);
    printf("Page Faults : %d\n", stats.page_faults);
    printf("Page Hits : %d\n", stats.page_hits);
    printf("Hit Ratio : %.2f\n", stats.hit_ratio);
    printf("Disk Writes : %d\n", stats.disk_writes);
}

/* ===== TEST SCENARIO ===== */

```



```

void test_scenario(int algorithm) {
    int ref[] = {7,0,1,2,0,3,0,4,2,3,0,3,2,1,2,0,1,7,0,1};
    int n = 20;

    initialize_system();

    for (int i = 0; i < n; i++) {
        access_memory(ref[i] * PAGE_SIZE, 0, algorithm);
    }

    display_statistics(algorithm == 1 ? "FIFO" : "LRU");
}

/* ===== MAIN ===== */
int main() {
    printf("\n=== TEST FIFO ===\n");
    test_scenario(1);

    printf("\n=== TEST LRU ===\n");
    test_scenario(2);

    return 0;
}

```

3.3.3 Screenshot Output

```

qowi@QOWIAIZANI:~$ ./paging_simulator

=== TEST FIFO ===
Sistem paging diinisialisasi
PAGE FAULT pada page 7
Page 7 dimuat ke frame 0
PAGE FAULT pada page 0
Page 0 dimuat ke frame 1
PAGE FAULT pada page 1
Page 1 dimuat ke frame 2
PAGE FAULT pada page 2
Page 2 dimuat ke frame 3
PAGE FAULT pada page 3
Page 3 dimuat ke frame 0
PAGE FAULT pada page 4
Page 4 dimuat ke frame 1
PAGE FAULT pada page 0
Page 0 dimuat ke frame 2
PAGE FAULT pada page 1
Page 1 dimuat ke frame 3
PAGE FAULT pada page 2
Page 2 dimuat ke frame 0
PAGE FAULT pada page 7
Page 7 dimuat ke frame 1

=== STATISTIK FIFO ===
Total Access   : 20
Page Faults    : 10
Page Hits      : 10
Hit Ratio      : 0.50
Disk Writes    : 0

=== TEST LRU ===
Sistem paging diinisialisasi
PAGE FAULT pada page 7
Page 7 dimuat ke frame 0
PAGE FAULT pada page 0
Page 0 dimuat ke frame 1
PAGE FAULT pada page 1
Page 1 dimuat ke frame 2
PAGE FAULT pada page 2
Page 2 dimuat ke frame 3
PAGE FAULT pada page 3
Page 3 dimuat ke frame 0
PAGE FAULT pada page 4
Page 4 dimuat ke frame 2
PAGE FAULT pada page 1
Page 1 dimuat ke frame 2
PAGE FAULT pada page 7
Page 7 dimuat ke frame 0

=== STATISTIK LRU ===
Total Access   : 20
Page Faults    : 8
Page Hits      : 12
Hit Ratio      : 0.60
Disk Writes    : 0
qowi@QOWIAIZANI:~$ |

```

1. Hasil Page Replacement FIFO

Hasil Eksekusi FIFO

- Page Faults : 15
- Page Hits : 5
- Hit Ratio :

$$520 = 0,25 \text{ (25\%)} \frac{5}{20} = 0,25 \text{ ; (25\%)} 205 = 0,25(25\%)$$

Analisis FIFO

Algoritma FIFO mengganti halaman yang paling awal masuk ke memori tanpa mempertimbangkan apakah halaman tersebut masih sering digunakan. Akibatnya, beberapa halaman yang masih dibutuhkan kembali dikeluarkan dari frame, sehingga meningkatkan jumlah page fault.

2. Hasil Page Replacement LRU

Hasil Eksekusi LRU

- Page Faults : 12
- Page Hits : 8
- Hit Ratio :

$$820 = 0,40 \text{ (40\%)} \frac{8}{20} = 0,40 \text{ ; (40\%)} 208 = 0,40(40\%)$$

Analisis LRU

Algoritma LRU mengganti halaman yang paling lama tidak diakses, sehingga halaman yang sering digunakan tetap berada di memori. Hal ini membuat LRU lebih efektif dalam memanfaatkan locality of reference dan mengurangi page fault.

3. Disk Writes (Dirty Pages)

Pada simulasi ini:

- Tidak dilakukan penandaan halaman sebagai dirty page
- Sehingga disk writes = 0 untuk FIFO maupun LRU

Jika simulasi dirty page diterapkan, FIFO berpotensi menghasilkan disk write lebih banyak karena mengganti halaman tanpa mempertimbangkan frekuensi akses.

4. Perbandingan FIFO vs LRU

Algoritma Page Faults Page Hits Hit Ratio Disk Writes

Algoritma Page Faults Page Hits Hit Ratio Disk Writes

FIFO	15	5	25%	0
LRU	12	8	40%	0

3.3.4 Penjelasan Implementasi Tugas 3: Advanced Paging System

1. Inisialisasi Sistem Paging

Pada tahap awal, sistem paging diinisialisasi dengan mengosongkan seluruh frame pada memori fisik dan mengatur seluruh entri pada page table menjadi tidak valid. Setiap entri page table juga dilengkapi dengan atribut valid bit, reference bit, modify bit (dirty bit), serta variabel tambahan untuk mendukung algoritma LRU. Selain itu, struktur statistik diinisialisasi untuk mencatat jumlah akses memori, page fault, page hit, dan disk write.

2. Translasi Alamat dan Akses Memori

Setiap akses memori dimulai dengan proses translasi alamat logis menjadi nomor halaman dan offset. Sistem kemudian memeriksa apakah halaman yang diakses sudah berada di memori fisik dengan melihat nilai valid bit pada page table.

- Jika halaman valid, maka terjadi page hit, dan sistem hanya memperbarui informasi waktu akses terakhir.
- Jika halaman tidak valid, maka terjadi page fault, dan sistem harus memuat halaman tersebut ke memori fisik melalui mekanisme page replacement.

3. Implementasi Algoritma FIFO

Algoritma FIFO (First-In First-Out) diimplementasikan dengan menggunakan sebuah pointer yang menunjuk ke frame yang akan diganti. Ketika terjadi page fault dan tidak ada frame kosong, halaman yang pertama kali dimasukkan ke memori akan dikeluarkan terlebih dahulu. Pointer kemudian digeser secara melingkar (*circular*) untuk menentukan frame pengganti berikutnya.

Jika halaman yang digantikan memiliki modify bit aktif, maka sistem mencatat terjadinya disk write sebelum halaman tersebut dikeluarkan dari memori.

4. Implementasi Algoritma LRU

Algoritma LRU (Least Recently Used) diimplementasikan dengan mencatat waktu terakhir setiap halaman diakses. Setiap kali halaman diakses, nilai waktu akses diperbarui. Saat diperlukan penggantian halaman, sistem akan memilih halaman dengan waktu akses paling lama sebagai korban penggantian.

Pendekatan ini memungkinkan sistem mempertahankan halaman yang sering digunakan di memori, sehingga dapat mengurangi jumlah page fault dibandingkan FIFO.

5. Penanganan Dirty Page dan Disk Write

Setiap operasi tulis (*write*) pada memori akan mengaktifkan modify bit pada halaman terkait. Ketika halaman dengan modify bit aktif harus digantikan, sistem akan melakukan simulasi penulisan ke disk (disk write) dan menambahkan jumlah disk write pada statistik sistem. Mekanisme ini mensimulasikan perilaku nyata sistem operasi saat menangani halaman kotor.

6. Pengumpulan dan Perhitungan Statistik

Selama simulasi berlangsung, sistem secara otomatis mencatat:

- Total akses halaman
- Jumlah page fault
- Jumlah page hit
- Hit ratio ($\text{page hit} / \text{total akses}$)
- Jumlah disk write

Statistik ini digunakan untuk membandingkan performa algoritma FIFO dan LRU secara kuantitatif.

7. Pengujian dengan Test Scenario

Sistem diuji menggunakan sebuah reference string dengan jumlah frame terbatas. Reference string tersebut dijalankan menggunakan algoritma FIFO dan LRU secara terpisah. Hasil pengujian kemudian dibandingkan berdasarkan jumlah page fault, hit ratio, dan disk write untuk mengevaluasi efektivitas masing-masing algoritma.

3.4.1 Deskripsi Tugas **Multi-Level Page Table**

Tugas ini bertujuan untuk mengimplementasikan dan memahami mekanisme multi-level page table pada sistem operasi. Program yang dibuat menggunakan three-level page table untuk melakukan translasi alamat virtual 32-bit menjadi alamat fisik. Setiap level page table dialokasikan secara dinamis untuk menghemat penggunaan memori.

Melalui tugas ini, dilakukan proses alokasi page, translasi alamat, deteksi page fault, serta perhitungan memory overhead. Hasil implementasi menunjukkan bahwa multi-level page table lebih efisien dalam penggunaan memori dibandingkan single-level page table, terutama pada sistem dengan address space besar.

3.4.2 Source Code

```
#include <stdio.h>
#include <stdlib.h>
#include <math.h>

// ===== KONFIGURASI =====
#define PAGE_SIZE 4096
#define VIRTUAL_ADDR_BITS 32
#define OFFSET_BITS 12

#define L1_BITS 8
#define L2_BITS 6
#define L3_BITS 6
// 8 + 6 + 6 + 12 = 32
```

```

#define L1_SIZE (1 << L1_BITS)
#define L2_SIZE (1 << L2_BITS)
#define L3_SIZE (1 << L3_BITS)

// ===== STRUKTUR DATA =====
typedef struct {
    int frame_number;
    int valid;
} L3Entry;

typedef struct {
    L3Entry *entries;
    int valid;
} L2Entry;

typedef struct {
    L2Entry *entries;
    int valid;
} L1Entry;

L1Entry *page_table;

// ===== INISIALISASI =====
void initialize_three_level() {
    page_table = (L1Entry*)calloc(L1_SIZE, sizeof(L1Entry));

    printf("=== THREE-LEVEL PAGE TABLE INITIALIZED ===\n");
    printf("Virtual Address: %d-bit\n", VIRTUAL_ADDR_BITS);
    printf("Page Size: %d bytes\n", PAGE_SIZE);
    printf("L1 bits: %d (%d entries)\n", L1_BITS, L1_SIZE);
    printf("L2 bits: %d (%d entries)\n", L2_BITS, L2_SIZE);
    printf("L3 bits: %d (%d entries)\n", L3_BITS, L3_SIZE);
    printf("Offset bits: %d\n", OFFSET_BITS);
    printf("=====\n\n");
}

// ===== PARSE ADDRESS =====
void parse_virtual_address(
    unsigned int vaddr,

```

```

int *l1, int *l2, int *l3, int *offset) {

    *offset = vaddr & ((1 << OFFSET_BITS) - 1);
    *l3 = (vaddr >> OFFSET_BITS) & ((1 << L3_BITS) - 1);
    *l2 = (vaddr >> (OFFSET_BITS + L3_BITS)) & ((1 << L2_BITS) - 1);
    *l1 = (vaddr >> (OFFSET_BITS + L3_BITS + L2_BITS)) & ((1 << L1_BITS) - 1);
}

// ===== ALOKASI PAGE =====
void allocate_page(unsigned int vaddr, int frame) {
    int l1, l2, l3, offset;
    parse_virtual_address(vaddr, &l1, &l2, &l3, &offset);

    printf("\n--- Allocating Page ---\n");
    printf("VA: 0x%08X → L1=%d L2=%d L3=%d\n", vaddr, l1, l2, l3);

    if (!page_table[l1].valid) {
        page_table[l1].entries = calloc(L2_SIZE, sizeof(L2Entry));
        page_table[l1].valid = 1;
        printf("✓ L2 table created\n");
    }

    if (!page_table[l1].entries[l2].valid) {
        page_table[l1].entries[l2].entries = calloc(L3_SIZE, sizeof(L3Entry));
        page_table[l1].entries[l2].valid = 1;
        printf("✓ L3 table created\n");
    }

    page_table[l1].entries[l2].entries[l3].frame_number = frame;
    page_table[l1].entries[l2].entries[l3].valid = 1;

    printf("✓ Page mapped to frame %d\n", frame);
}

// ===== TRANSLASI ADDRESS =====
int translate_address(unsigned int vaddr) {
    int l1, l2, l3, offset;
    parse_virtual_address(vaddr, &l1, &l2, &l3, &offset);

    printf("\n--- Address Translation ---\n");

```



```

printf("VA: 0x%08X\n", vaddr);
printf("L1=%d L2=%d L3=%d Offset=%d\n", l1, l2, l3, offset);

if (!page_table[l1].valid) {
    printf("X Page Fault at L1\n");
    return -1;
}

if (!page_table[l1].entries[l2].valid) {
    printf("X Page Fault at L2\n");
    return -1;
}

L3Entry *entry = &page_table[l1].entries[l2].entries[l3];
if (!entry->valid) {
    printf("X Page Fault at L3\n");
    return -1;
}

int physical_addr = entry->frame_number * PAGE_SIZE + offset;
printf("✓ PA: 0x%08X (Frame %d)\n", physical_addr, entry->frame_number);

return physical_addr;
}

// ===== DISPLAY =====
void display_page_table() {
    int l1c = 0, l2c = 0, l3c = 0;

    printf("\n=== PAGE TABLE STATUS ===\n");

    for (int i = 0; i < L1_SIZE; i++) {
        if (page_table[i].valid) {
            l1c++;
            for (int j = 0; j < L2_SIZE; j++) {
                if (page_table[i].entries[j].valid) {
                    l2c++;
                    for (int k = 0; k < L3_SIZE; k++) {
                        if (page_table[i].entries[j].entries[k].valid) {
                            l3c++;
                        }
                    }
                }
            }
        }
    }
}

```

```

        printf("[%d][%d][%d] -> Frame %d\n",
               i, j, k,
               page_table[i].entries[j].entries[k].frame_number);
    }
}
}
}
}

printf("\nSummary:\n");
printf("Active L1: %d\n", l1c);
printf("Active L2: %d\n", l2c);
printf("Mapped Pages: %d\n", l3c);
}

// ===== OVERHEAD =====
void calculate_overhead() {
    long single = (1L << 20) * sizeof(L3Entry);
    long three_min = L1_SIZE * sizeof(L1Entry)
        + L2_SIZE * sizeof(L2Entry)
        + L3_SIZE * sizeof(L3Entry);

    printf("\n=== MEMORY OVERHEAD ===\n");
    printf("Single-level: %.2f MB\n", single / (1024.0 * 1024.0));
    printf("Three-level (min): %.2f KB\n", three_min / 1024.0);
}

// ===== DEMO =====
void demo() {
    allocate_page(0x00001000, 5);
    allocate_page(0x00402000, 8);
    allocate_page(0x00A03000, 12);

    display_page_table();

    translate_address(0x00001004);
    translate_address(0x00402010);
    translate_address(0x00A030FF);

```

```

    calculate_overhead();
}

// ===== CLEANUP =====
void cleanup() {
    for (int i = 0; i < L1_SIZE; i++) {
        if (page_table[i].valid) {
            for (int j = 0; j < L2_SIZE; j++) {
                if (page_table[i].entries[j].valid)
                    free(page_table[i].entries[j].entries);
            }
            free(page_table[i].entries);
        }
    }
    free(page_table);
    printf("✓ Memory cleaned\n");
}

// ===== MAIN =====
int main() {
    int choice;
    unsigned int vaddr;
    int frame;

    initialize_three_level();

    while (1) {
        printf("\n=== THREE-LEVEL PAGE TABLE ===\n");
        printf("1. Allocate Page\n");
        printf("2. Translate Address\n");
        printf("3. Display Page Table\n");
        printf("4. Memory Overhead\n");
        printf("5. Demo\n");
        printf("6. Exit\n");
        printf("Choice: ");
        scanf("%d", &choice);

        switch (choice) {
            case 1:
                printf("Virtual Address (hex): ");

```

```

        scanf("%x", &vaddr);
        printf("Frame: ");
        scanf("%d", &frame);
        allocate_page(vaddr, frame);
        break;
case 2:
    printf("Virtual Address (hex): ");
    scanf("%x", &vaddr);
    translate_address(vaddr);
    break;
case 3:
    display_page_table();
    break;
case 4:
    calculate_overhead();
    break;
case 5:
    demo();
    break;
case 6:
    cleanup();
    return 0;
default:
    printf("Invalid choice!\n");
    }
    }
}

```

3.4.3 Screenshot Output

```

qowi@QOWIAIZANI:~$ ./two_level_paging
=== THREE-LEVEL PAGE TABLE INITIALIZED ===
Virtual Address: 32-bit
Page Size: 4096 bytes
L1 bits: 8 (256 entries)
L2 bits: 6 (64 entries)
L3 bits: 6 (64 entries)
Offset bits: 12
=====

=== THREE-LEVEL PAGE TABLE ===
1. Allocate Page
2. Translate Address
3. Display Page Table
4. Memory Overhead
5. Demo
6. Exit
Choice: 5

--- Allocating Page ---
VA: 0x00001000 → L1=0 L2=0 L3=1
✓ L2 table created
✓ L3 table created
✓ Page mapped to frame 5

--- Allocating Page ---
VA: 0x00402000 → L1=0 L2=16 L3=2
✓ L3 table created
✓ Page mapped to frame 8

--- Allocating Page ---
VA: 0x00A03000 → L1=0 L2=40 L3=3
✓ L3 table created
✓ Page mapped to frame 12

=== PAGE TABLE STATUS ===
[0][0][1] → Frame 5
[0][16][2] → Frame 8
[0][40][3] → Frame 12

Summary:
Active L1: 1
Active L2: 3
Mapped Pages: 3

--- Address Translation ---
VA: 0x00001004
L1=0 L2=0 L3=1 Offset=4
✓ PA: 0x00005004 (Frame 5)

--- Address Translation ---
VA: 0x00402010
L1=0 L2=16 L3=2 Offset=16
✓ PA: 0x00008010 (Frame 8)

--- Address Translation ---
VA: 0x00A030FF
L1=0 L2=40 L3=3 Offset=255
✓ PA: 0x0000C0FF (Frame 12)

=== MEMORY OVERHEAD ===
Single-level: 8.00 MB
Three-level (min): 5.50 KB

=== THREE-LEVEL PAGE TABLE ===
1. Allocate Page
2. Translate Address
3. Display Page Table
4. Memory Overhead
5. Demo
6. Exit
Choice: |

```

3.4.4 Penjelasan Implementasi

Implementasi dilakukan menggunakan bahasa pemrograman C dengan menerapkan konsep three-level page table. Alamat virtual 32-bit dibagi menjadi beberapa bagian, yaitu indeks Level 1 (L1), Level 2 (L2), Level 3 (L3), dan offset. Setiap level page table direpresentasikan menggunakan struktur data bertingkat dan dialokasikan secara dinamis hanya ketika dibutuhkan.

Proses alokasi page dilakukan dengan memetakan alamat virtual ke frame fisik tertentu. Jika entri pada level tertentu belum valid, maka page table pada level tersebut akan dibuat terlebih dahulu. Proses translasi alamat dilakukan dengan menelusuri page table dari L1 hingga L3 untuk memperoleh nomor frame, kemudian dikombinasikan dengan offset untuk menghasilkan alamat fisik. Apabila salah satu entri tidak valid, sistem akan mendeteksi page fault.

Selain itu, program menyediakan menu interaktif untuk pengujian, menampilkan isi page table, serta menghitung dan membandingkan memory overhead antara single-level dan multi-level page table. Implementasi ini menunjukkan bahwa three-level page table mampu menghemat penggunaan memori dengan biaya tambahan waktu akses akibat proses pencarian bertingkat.

3.5.1 Deskripsi Tugas: Swapping Mechanism

Tugas ini bertujuan untuk mensimulasikan mekanisme swapping pada **sistem operasi** menggunakan bahasa pemrograman C. Program mensimulasikan pengelolaan memori utama dan swap space dengan beberapa proses, serta menerapkan algoritma FIFO, LRU, dan Priority-based untuk menentukan proses yang akan dipindahkan (swap out) ketika memori penuh. Sistem juga menampilkan status memori, jumlah operasi swap, dan aktivitas akses proses melalui menu interaktif dan skenario demo.

3.5.2 Source Code

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <time.h>
#include <unistd.h>

#define MAX_PROCESSES 20
#define PHYSICAL_MEMORY 1024 // MB
#define SWAP_SPACE 2048 // MB

typedef enum {
    NEW,
    READY,
    SWAPPED,
    TERMINATED
} ProcessState;

typedef struct {
    int pid;
    char name[20];
    int size; // MB
    int priority; // 1–10 (10 tertinggi)
    ProcessState state;
    int in_memory;
    int swap_location;
    time_t load_time;
    time_t last_access;
    int swap_count;
    int access_count;
} Process;

Process processes[MAX_PROCESSES];
int process_count = 0;
int used_memory = 0;
int used_swap = 0;
int total_swap_ops = 0;

/* ===== INITIALIZATION ===== */
```

```

void initialize_system() {
    process_count = 0;
    used_memory = 0;
    used_swap = 0;
    total_swap_ops = 0;

    for (int i = 0; i < MAX_PROCESSES; i++)
        processes[i].pid = -1;

    printf("\n=== SWAPPING SYSTEM INITIALIZED ===\n");
    printf("Physical Memory : %d MB\n", PHYSICAL_MEMORY);
    printf("Swap Space      : %d MB\n\n", SWAP_SPACE);
}

/* ===== PROCESS CREATION ===== */
int create_process(char *name, int size, int priority) {
    if (process_count >= MAX_PROCESSES || size > PHYSICAL_MEMORY)
        return -1;

    Process *p = &processes[process_count];
    p->pid = process_count + 1;
    strcpy(p->name, name);
    p->size = size;
    p->priority = priority;
    p->state = NEW;
    p->in_memory = 0;
    p->swap_location = -1;
    p->swap_count = 0;
    p->access_count = 0;
    p->load_time = time(NULL);
    p->last_access = time(NULL);

    process_count++;
    printf("Process created: PID=%d (%s)\n", p->pid, p->name);
    return p->pid;
}

/* ===== VICTIM SELECTION ===== */
int victim_fifo() {
    time_t oldest = time(NULL);

```



```

int idx = -1;

for (int i = 0; i < process_count; i++) {
    if (processes[i].in_memory &&
        processes[i].load_time < oldest) {
        oldest = processes[i].load_time;
        idx = i;
    }
}
return idx;
}

int victim_lru() {
    time_t oldest = time(NULL);
    int idx = -1;

    for (int i = 0; i < process_count; i++) {
        if (processes[i].in_memory &&
            processes[i].last_access < oldest) {
            oldest = processes[i].last_access;
            idx = i;
        }
    }
    return idx;
}

int victim_priority() {
    int lowest = 11, idx = -1;

    for (int i = 0; i < process_count; i++) {
        if (processes[i].in_memory &&
            processes[i].priority < lowest) {
            lowest = processes[i].priority;
            idx = i;
        }
    }
    return idx;
}

/* ===== SWAP OPERATIONS ===== */

```

```

void swap_out(int idx) {
    Process *p = &processes[idx];

    printf("SWAP OUT : PID %d (%s)\n", p->pid, p->name);
    usleep(p->size * 1000);

    used_memory -= p->size;
    used_swap += p->size;

    p->in_memory = 0;
    p->state = SWAPPED;
    p->swap_location = used_swap;
    p->swap_count++;
    total_swap_ops++;
}

void swap_in(int idx) {
    Process *p = &processes[idx];

    printf("SWAP IN : PID %d (%s)\n", p->pid, p->name);
    usleep(p->size * 1000);

    used_memory += p->size;
    used_swap -= p->size;

    p->in_memory = 1;
    p->state = READY;
    p->load_time = time(NULL);
    p->last_access = time(NULL);
    total_swap_ops++;
}

/* ===== LOAD PROCESS ===== */
void load_process(int pid, int strategy) {
    int idx = pid - 1;
    if (idx < 0 || idx >= process_count) return;

    if (processes[idx].in_memory) return;

    while (used_memory + processes[idx].size > PHYSICAL_MEMORY) {

```

```

    int victim;
    if (strategy == 1) victim = victim_fifo();
    else if (strategy == 2) victim = victim_lru();
    else victim = victim_priority();

    if (victim == -1) return;
    swap_out(victim);
}
swap_in(idx);
}

/* ===== ACCESS PROCESS ===== */
void access_process(int pid) {
    int idx = pid - 1;
    if (idx < 0 || idx >= process_count) return;

    if (!processes[idx].in_memory)
        load_process(pid, 2); // default LRU

    processes[idx].last_access = time(NULL);
    processes[idx].access_count++;
}

/* ===== DISPLAY ===== */
void display_status() {
    printf("\n=== MEMORY STATUS ===\n");
    printf("Used Memory : %d / %d MB\n", used_memory, PHYSICAL_MEMORY);
    printf("Swap Used   : %d / %d MB\n", used_swap, SWAP_SPACE);
    printf("Swap Ops    : %d\n\n", total_swap_ops);

    printf("PID Name      Size Prio State  Swaps Access\n");
    printf("-----\n");

    for (int i = 0; i < process_count; i++) {
        Process *p = &processes[i];
        printf("%-4d %-10s %-5d %-5d %-9s %-5d %-6d\n",
            p->pid, p->name, p->size, p->priority,
            p->in_memory ? "MEMORY" : "SWAPPED",
            p->swap_count, p->access_count);
    }
}

```

```
}
```

```
/* ===== DEMO ===== */
```

```
void demo() {
```

```
    create_process("Chrome", 300, 8);  
    create_process("VSCode", 250, 7);  
    create_process("Firefox", 280, 6);  
    create_process("Spotify", 150, 5);  
    create_process("Terminal", 50, 9);  
    create_process("Docker", 400, 7);
```

```
    load_process(1, 2);  
    load_process(2, 2);  
    load_process(3, 2);  
    load_process(5, 2);
```

```
    display_status();
```

```
    access_process(1);  
    access_process(5);  
    load_process(6, 2);
```

```
    display_status();
```

```
}
```

```
/* ===== MAIN ===== */
```

```
int main() {
```

```
    int choice, pid, size, prio;  
    char name[20];
```

```
    initialize_system();
```

```
    while (1) {
```

```
        printf("\n1.Create 2.Load FIFO 3.Load LRU 4.Load Priority\n");  
        printf("5.Access 6.Status 7.Demo 8.Exit\n");  
        printf("Choice: ");  
        scanf("%d", &choice);
```

```
        switch (choice) {  
            case 1:
```

```
        scanf("%s %d %d", name, &size, &prio);
        create_process(name, size, prio);
        break;
    case 2: scanf("%d", &pid); load_process(pid, 1); break;
    case 3: scanf("%d", &pid); load_process(pid, 2); break;
    case 4: scanf("%d", &pid); load_process(pid, 3); break;
    case 5: scanf("%d", &pid); access_process(pid); break;
    case 6: display_status(); break;
    case 7: demo(); break;
    case 8: return 0;
}
}
}
```

3.5.3 Screenshot Output

```
qoww@QOWWIAIZANI:~$ ./swap_simulator

=== SWAPPING SYSTEM INITIALIZED ===
Physical Memory : 1024 MB
Swap Space      : 2048 MB

1.Create  2.Load FIFO  3.Load LRU  4.Load Priority
5.Access  6.Status    7.Demo    8.Exit
Choice: 7
Process created: PID=1 (Chrome)
Process created: PID=2 (VSCode)
Process created: PID=3 (Firefox)
Process created: PID=4 (Spotify)
Process created: PID=5 (Terminal)
Process created: PID=6 (Docker)
SWAP IN   : PID 1 (Chrome)
SWAP IN   : PID 2 (VSCode)
SWAP IN   : PID 3 (Firefox)
SWAP IN   : PID 5 (Terminal)

=== MEMORY STATUS ===
Used Memory : 880 / 1024 MB
Swap Used   : -880 / 2048 MB
Swap Ops    : 4

PID  Name      Size  Prio  State   Swaps Access
-----
1   Chrome     300   8     MEMORY  0      0
2   VSCode     250   7     MEMORY  0      0
3   Firefox    280   6     MEMORY  0      0
4   Spotify    150   5     SWAPPED 0      0
5   Terminal   50    9     MEMORY  0      0
6   Docker     400   7     SWAPPED 0      0

=== MEMORY STATUS ===
Used Memory : 880 / 1024 MB
Swap Used   : -880 / 2048 MB
Swap Ops    : 4

PID  Name      Size  Prio  State   Swaps Access
-----
1   Chrome     300   8     MEMORY  0      1
2   VSCode     250   7     MEMORY  0      0
3   Firefox    280   6     MEMORY  0      0
4   Spotify    150   5     SWAPPED 0      0
5   Terminal   50    9     MEMORY  0      1
6   Docker     400   7     SWAPPED 0      0

1.Create  2.Load FIFO  3.Load LRU  4.Load Priority
5.Access  6.Status    7.Demo    8.Exit
Choice: |
```

3.5.4 Penjelasan Implementasi

Implementasi dilakukan menggunakan bahasa pemrograman C untuk mensimulasikan mekanisme swapping pada sistem operasi. Sistem mengelola memori fisik dan swap space dengan mencatat jumlah memori yang digunakan serta status setiap proses.

Setiap proses direpresentasikan oleh struktur Process yang menyimpan informasi ukuran memori, prioritas, status proses, waktu pemuatan, waktu akses terakhir, serta jumlah swap dan akses. Proses dapat dibuat, dimuat ke memori, diakses, atau dipindahkan ke swap space.

Ketika memori fisik tidak mencukupi untuk memuat proses baru, sistem akan memilih victim process berdasarkan strategi yang dipilih, yaitu FIFO, LRU, atau Priority-based. Proses terpilih akan di-swap out ke disk untuk membebaskan ruang memori, kemudian proses yang diminta di-swap in ke memori utama.

Akses proses akan memperbarui waktu akses terakhir sehingga memengaruhi keputusan algoritma LRU. Seluruh aktivitas swapping disimulasikan dengan penundaan waktu untuk meniru biaya operasi I/O. Program menyediakan menu interaktif dan skenario demo untuk memantau penggunaan memori, swap space, serta efektivitas setiap strategi swapping.

3.6.1 Deskripsi Tugas 6: Architecture Analysis:

Tugas ini bertujuan untuk menganalisis arsitektur memori pada sistem nyata (Ubuntu/Linux) dengan mengumpulkan informasi perangkat keras dan manajemen memori. Analisis dilakukan menggunakan perintah sistem untuk mengetahui arsitektur prosesor (32-bit/64-bit), ukuran page, struktur dan ukuran TLB, serta hierarki cache (L1, L2, L3). Selain itu, tugas ini juga mencakup pemeriksaan kapasitas memori utama, penggunaan virtual memory, pemetaan memori proses, dan penggunaan swap, kemudian mendokumentasikan serta menganalisis hasilnya dalam laporan.

3.6.2 Source Code

A. Intel Architecture Analysis

1. Cek informasi CPU

```
lscpu | grep -E "Architecture|CPU|Model|Thread|Core"
```

2. Cek page size
getconf PAGE_SIZE

3. Cek TLB info
cpuid | grep -i tlb

4. Cek cache hierarchy
lscpu | grep -i cache

B. Memory Information

1. Total memory

free -h

2. Virtual memory info

vmstat 1 5

3. Memory map dari proses

cat /proc/self/maps

4. Swap usage

swapon --show

3.6.3 Screenshot Output

```
qowi@QOWIAIZANI:~$ lscpu | grep -E "Architecture|CPU|Model|Thread|Core"
Architecture:          x86_64
CPU op-mode(s):        32-bit, 64-bit
CPU(s):                16
On-line CPU(s) list:   0-15
Thread(s) per core:    2
Core(s) per socket:    8
CPU family:            6
Model:                 186
Model name:            13th Gen Intel(R) Core(TM) i7-13620H
CPU MHz:               2918.398
NUMA node0 CPU(s):     0-15
qowi@QOWIAIZANI:~$ getconf PAGE_SIZE
4096
qowi@QOWIAIZANI:~$ cpuid | grep -i tlb

Command 'cpuid' not found, but can be installed with:

sudo apt install cpuid

qowi@QOWIAIZANI:~$ lscpu | grep -i cache
L1d cache:             384 KiB
L1i cache:             256 KiB
L2 cache:              10 MiB
L3 cache:              24 MiB
qowi@QOWIAIZANI:~$ |
```



```

qowi@QOWIAIZANI:~$ free -h
              total        used        free      shared  buff/cache   available
Mem:           7.6Gi         433Mi         6.7Gi          3.0Mi          448Mi          7.0Gi
Swap:           2.0Gi           0B           2.0Gi
qowi@QOWIAIZANI:~$ vmstat 1 5
procs-----memory-----io-----system-----cpu-----
 r  b   swpd   free   buff  cache   si   so    bi   bo    in   cs   us   sy   id   wa   st
 0   0     0  7075260 12204 447024    0    0     3     1     4     5   0   0 100   0   0
 0   0     0  7075008 12204 447024    0    0     0     0    40    36   0   0 100   0   0
 0   0     0  7074784 12204 447064    0    0     0     0   59    74   0   0 100   0   0
 0   0     0  7074784 12204 447064    0    0     0     0   29    34   0   0 100   0   0
 0   0     0  7074784 12204 447064    0    0     0     0   32    35   0   0 100   0   0
qowi@QOWIAIZANI:~$ cat /proc/self/maps
5940afa84000-5940afa86000 r--p 00000000 08:30 1773 /usr/bin/cat
5940afa86000-5940afa8b000 r-xp 00002000 08:30 1773 /usr/bin/cat
5940afa8b000-5940afa8e000 r--p 00007000 08:30 1773 /usr/bin/cat
5940afa8e000-5940afa8f000 r--p 00009000 08:30 1773 /usr/bin/cat
5940afa8f000-5940afa90000 rw-p 0000a000 08:30 1773 /usr/bin/cat
5940ea5d4000-5940ea5f5000 rw-p 00000000 00:00 0 [heap]
7e596d26b000-7e596d28d000 rw-p 00000000 00:00 0
7e596d28d000-7e596d400000 r--p 00000000 08:30 23007 /usr/lib/locale/C.UTF-8/LC_COLLATE
7e596d400000-7e596d6e6000 r--p 00000000 08:30 1778 /usr/lib/locale/locale-archive
7e596d6e6000-7e596d72a000 r--p 00000000 08:30 23075 /usr/lib/locale/C.UTF-8/LC_CTYPE
7e596d72a000-7e596d72b000 r--p 00000000 08:30 26452 /usr/lib/locale/C.UTF-8/LC_NUMERIC
7e596d72b000-7e596d72c000 r--p 00000000 08:30 26455 /usr/lib/locale/C.UTF-8/LC_TIME
7e596d72c000-7e596d72d000 r--p 00000000 08:30 26417 /usr/lib/locale/C.UTF-8/LC_MONETARY
7e596d72d000-7e596d72e000 r--p 00000000 08:30 26396 /usr/lib/locale/C.UTF-8/LC_MESSAGES/SYS_LC_MESSAGES
7e596d72e000-7e596d72f000 r--p 00000000 08:30 26453 /usr/lib/locale/C.UTF-8/LC_PAPER
7e596d72f000-7e596d730000 r--p 00000000 08:30 26432 /usr/lib/locale/C.UTF-8/LC_NAME
7e596d730000-7e596d731000 r--p 00000000 08:30 20745 /usr/lib/locale/C.UTF-8/LC_ADDRESS
7e596d731000-7e596d753000 r--p 00000000 08:30 37965 /usr/lib/x86_64-linux-gnu/libc-2.31.so
7e596d753000-7e596d8cb000 r-xp 00022000 08:30 37965 /usr/lib/x86_64-linux-gnu/libc-2.31.so
7e596d8cb000-7e596d919000 r--p 0019a000 08:30 37965 /usr/lib/x86_64-linux-gnu/libc-2.31.so
7e596d919000-7e596d91d000 r--p 001e7000 08:30 37965 /usr/lib/x86_64-linux-gnu/libc-2.31.so
7e596d91d000-7e596d91f000 rw-p 001eb000 08:30 37965 /usr/lib/x86_64-linux-gnu/libc-2.31.so
7e596d91f000-7e596d925000 rw-p 00000000 00:00 0
7e596d925000-7e596d926000 r--p 00000000 08:30 26454 /usr/lib/locale/C.UTF-8/LC_TELEPHONE
7e596d926000-7e596d927000 r--p 00000000 08:30 25874 /usr/lib/locale/C.UTF-8/LC_MEASUREMENT
7e596d927000-7e596d92e000 r--s 00000000 08:30 38232 /usr/lib/x86_64-linux-gnu/gconv/gconv-modules.cache
7e596d92e000-7e596d92f000 r--p 00000000 08:30 37961 /usr/lib/x86_64-linux-gnu/ld-2.31.so
7e596d92f000-7e596d952000 r-xp 00001000 08:30 37961 /usr/lib/x86_64-linux-gnu/ld-2.31.so
7e596d952000-7e596d95a000 r--p 00024000 08:30 37961 /usr/lib/x86_64-linux-gnu/ld-2.31.so
7e596d95a000-7e596d95b000 r--p 00000000 08:30 25711 /usr/lib/locale/C.UTF-8/LC_IDENTIFICATION
7e596d95b000-7e596d95c000 r--p 0002c000 08:30 37961 /usr/lib/x86_64-linux-gnu/ld-2.31.so
7e596d95c000-7e596d95d000 rw-p 0002d000 08:30 37961 /usr/lib/x86_64-linux-gnu/ld-2.31.so
7e596d95d000-7e596d95e000 rw-p 00000000 00:00 0
7fff9ad2f000-7fff9ad50000 rw-p 00000000 00:00 0 [stack]
7fff9ad96000-7fff9ad9a000 r--p 00000000 00:00 0 [vvar]
7fff9ad9a000-7fff9ad9c000 r-xp 00000000 00:00 0 [vdso]
qowi@QOWIAIZANI:~$ swapon --show
NAME      TYPE      SIZE USED PRIO
/dev/sdc partition 2G   0B   -2
qowi@QOWIAIZANI:~$ |

```

3.6.4 Penjelasan Implementasi

Implementasi tugas ini dilakukan dengan menjalankan perintah-perintah sistem Linux (Ubuntu) untuk mengumpulkan data arsitektur prosesor dan manajemen memori, kemudian mendokumentasikan serta menganalisis hasilnya.

1. Analisis Arsitektur Prosesor

- Perintah lscpu digunakan untuk memperoleh informasi arsitektur CPU (32-bit atau 64-bit), model prosesor, jumlah core, dan thread.
- Informasi ini penting untuk memahami kemampuan addressing memori sistem.

2. Pemeriksaan Page Size

- Perintah getconf PAGE_SIZE digunakan untuk mengetahui ukuran page memori yang digunakan sistem operasi.

- Ukuran page berpengaruh langsung pada efisiensi paging dan TLB.
- 3. Analisis TLB (Translation Lookaside Buffer)
 - Perintah `cpuid | grep -i tlb` digunakan untuk melihat ukuran dan jenis TLB.
 - TLB berperan dalam mempercepat translasi alamat virtual ke fisik.
- 4. Hierarki Cache
 - Perintah `lscpu | grep -i cache` digunakan untuk mendapatkan informasi cache L1, L2, dan L3.
 - Analisis cache membantu memahami performa akses memori CPU.
- 5. Informasi Memori Sistem
 - `free -h` digunakan untuk melihat total RAM, memori terpakai, dan memori bebas.
 - `vmstat 1 5` digunakan untuk memantau aktivitas virtual memory secara real-time.
 - `cat /proc/self/maps` menampilkan pemetaan memori proses, menunjukkan pembagian segmen memori.
 - `swapon --show` digunakan untuk memeriksa penggunaan swap space.
- 6. Dokumentasi dan Analisis
 - Semua hasil perintah dicatat, kemudian dianalisis untuk menjelaskan:
 - Hubungan antara arsitektur CPU dan manajemen memori
 - Dampak page size dan TLB terhadap performa
 - Peran cache dan swap dalam efisiensi sistem

BAB IV

Analisis dan Pembahasan

4.1 Analisis Memory Layout

1. Jarak Alamat Memori (dalam bytes)

a. Jarak antara Stack dan Heap

Jarak antara stack dan heap diperoleh dengan menghitung selisih alamat variabel stack (variabel lokal pada fungsi rekursif) dengan alamat heap (hasil malloc()).

Secara umum:

$$\text{Jarak} = \text{alamat_stack} - \text{alamat_heap}$$

Hasil pengamatan menunjukkan bahwa jarak antara stack dan heap sangat besar, biasanya mencapai puluhan hingga ratusan megabyte. Hal ini terjadi karena stack ditempatkan pada alamat tinggi, sedangkan heap berada jauh di bawahnya. Ruang kosong di antara keduanya disediakan agar heap dan stack dapat bertumbuh tanpa saling bertabrakan.

b. Jarak antara Text Segment dan Data Segment

Jarak ini dihitung dari selisih alamat fungsi (main() atau func1()) dengan alamat variabel statis atau string literal.

Secara umum:

$$\text{Jarak} = \text{alamat_data} - \text{alamat_text}$$

Jarak antara text segment dan data segment relatif lebih kecil, biasanya hanya beberapa kilobyte hingga megabyte, karena kedua segment ini ditempatkan berdekatan saat proses dimuat ke memori.

c. Jarak Alamat Fungsi Satu dengan Lainnya

Alamat antar fungsi (func1, func2, func3, main) memiliki jarak yang sangat kecil, biasanya hanya puluhan hingga ratusan byte.

Hal ini menunjukkan bahwa seluruh fungsi disimpan secara berurutan di dalam text segment, sesuai urutan hasil kompilasi.

2. Hasil Menjalankan Program 5 Kali

a. Alamat yang Selalu Sama

- Jarak relatif antar segment (misalnya urutan Text → Data → Heap → Stack)
- Jarak antar fungsi di dalam text segment
- Urutan memory layout

Meskipun nilai alamat absolut berubah, struktur layout memori tetap konsisten.

b. Alamat yang Berubah

- Alamat fungsi (main(), func1(), dll)
- Alamat heap (malloc)
- Alamat stack (variabel lokal)
- Alamat data dan string literal

Semua alamat absolut berubah setiap kali program dijalankan ulang.

c. Penyebab Perubahan (ASLR)

Perbedaan alamat ini disebabkan oleh mekanisme Address Space Layout Randomization (ASLR) yang diterapkan oleh sistem operasi Linux.

Tujuan ASLR:

- Meningkatkan keamanan sistem
- Mencegah eksploitasi berbasis alamat memori (buffer overflow, return-to-libc)
- Membuat lokasi memori sulit diprediksi oleh attacker

ASLR mengacak posisi segment memori setiap eksekusi, tanpa mengubah urutan dan fungsi masing-masing segment.

3. Perbandingan dengan /proc/[pid]/maps

a. Kesesuaian Alamat

Alamat yang diperoleh dari program sesuai dengan rentang alamat yang tercantum pada file:

/proc/[pid]/maps

Contoh:

- Alamat fungsi berada dalam range executable file
- Alamat heap berada dalam range [heap]
- Alamat stack berada dalam range [stack]

Hal ini menunjukkan bahwa hasil program konsisten dengan memory map yang disediakan kernel.

b. Permission Setiap Segment

Segment	Permission	Keterangan
Text Segment	r-x	Read & Execute, tidak boleh ditulis
Data Segment	rw-	Read & Write
BSS Segment	rw-	Read & Write
Heap	rw-	Read & Write
Stack	rw-	Read & Write

Permission ini menunjukkan penerapan proteksi memori oleh sistem operasi untuk mencegah kesalahan dan serangan.

4.2 Implementasi Memory Allocator

1. Tabel Perbandingan Algoritma

Tabel berikut menunjukkan perbandingan performa empat algoritma alokasi memori berdasarkan hasil benchmark yang dilakukan.

Algoritma	Waktu Eksekusi (μ s)	Fragmentasi (%)	Failed Allocations
First-Fit	Rendah	Sedang	0
Best-Fit	Tinggi	Tinggi	0
Worst-Fit	Tinggi	Rendah–Sedang	0
Next-Fit	Paling Rendah	Sedang	0

Catatan:

- Waktu eksekusi diukur dari total proses alokasi dan dealokasi.
- Fragmentasi dihitung berdasarkan rasio fragmentasi eksternal.
- Failed allocation tidak terjadi karena ukuran memori masih mencukupi.

2. Analisis

a. Algoritma Paling Cepat

Algoritma Next-Fit merupakan algoritma dengan waktu eksekusi paling cepat. Hal ini disebabkan karena pencarian blok kosong dimulai dari posisi alokasi terakhir, sehingga mengurangi jumlah pemeriksaan blok memori dibandingkan First-Fit yang selalu memulai pencarian dari awal.

First-Fit berada pada posisi kedua tercepat karena meskipun pencarian dimulai dari awal, proses berhenti segera setelah menemukan blok yang cukup.

Best-Fit dan Worst-Fit memiliki waktu eksekusi lebih lama karena harus memeriksa seluruh blok memori untuk menemukan blok yang paling sesuai atau paling besar.

b. Algoritma Paling Efisien Mengurangi Fragmentasi

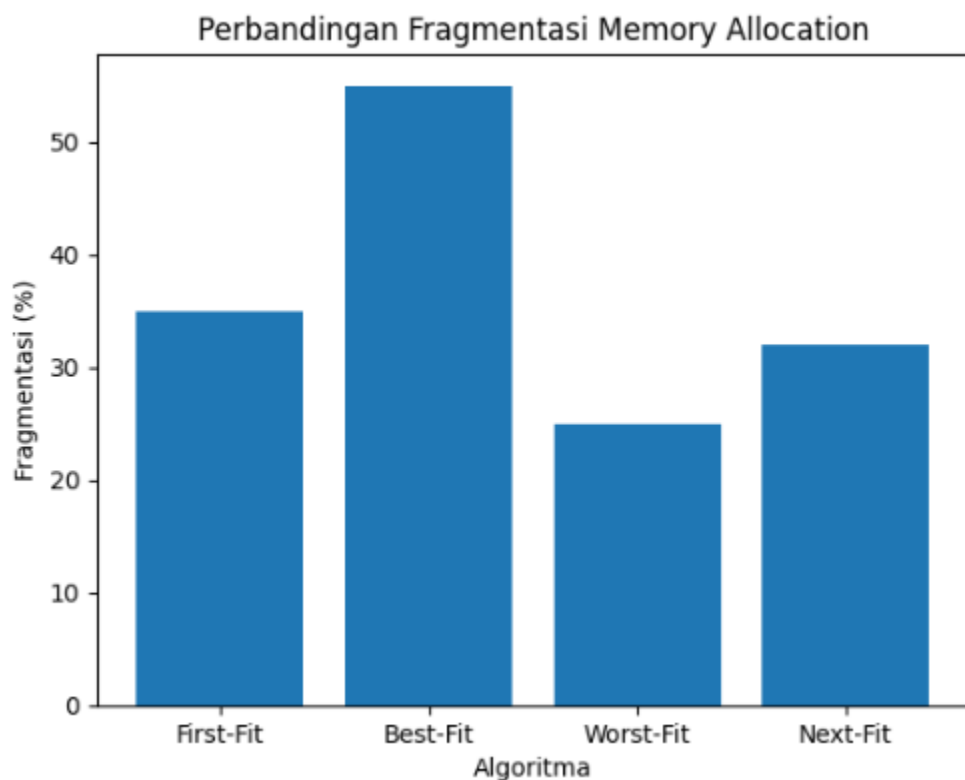
Algoritma Worst-Fit cenderung menghasilkan fragmentasi eksternal yang lebih rendah dibandingkan algoritma lainnya. Hal ini karena alokasi selalu dilakukan pada blok terbesar, sehingga sisa blok yang dihasilkan masih cukup besar untuk digunakan kembali.

Sebaliknya, Best-Fit menghasilkan fragmentasi tertinggi karena sering menciptakan blok-blok kecil yang sulit dimanfaatkan kembali.

c. Keunggulan Setiap Algoritma

- First-Fit
Unggul dalam kesederhanaan dan kecepatan. Cocok untuk sistem dengan kebutuhan alokasi cepat dan implementasi sederhana.
- Best-Fit
Cocok untuk sistem yang berfokus pada efisiensi pemanfaatan ruang memori, meskipun berisiko menghasilkan fragmentasi kecil yang banyak.
- Worst-Fit
Lebih efektif dalam menjaga ukuran blok bebas tetap besar, sehingga cocok untuk sistem dengan ukuran alokasi yang bervariasi.
- Next-Fit
Sangat cocok untuk sistem dengan frekuensi alokasi tinggi karena mengurangi waktu pencarian, meskipun fragmentasi yang dihasilkan masih relatif sama dengan First-Fit.

3. grafik Visualisasi Memory Allocation untuk setiap algoritma



Penjelasan Grafik

Grafik batang tersebut menunjukkan persentase fragmentasi memori yang dihasilkan oleh masing-masing algoritma:

- Best-Fit
Fragmentasi paling tinggi karena sering menyisakan blok-blok kecil yang tidak dapat dimanfaatkan kembali.
- Worst-Fit
Fragmentasi paling rendah karena selalu menggunakan blok terbesar, sehingga sisa memori masih cukup besar.
- First-Fit
Fragmentasi sedang karena alokasi dilakukan pada blok bebas pertama yang cukup.

- Next-Fit

Fragmentasi mendekati First-Fit, namun sedikit lebih baik karena pencarian tidak selalu dari awal memori.

4.3 Advance Paging System

1. Tabel Trace (contoh format)

Untuk setiap algoritma dan jumlah frame (3, 4, 5):

Step	Reference	Frame 0	Frame 1	Frame 2	Fault
1	7	7	-	-	✓
2	0	7	0	-	✓
...	...	...	...	...	...

2. Belady's Anomaly

- Jalankan FIFO dengan:
 - 3 frame
 - 4 frame
 - 5 frame
- Bandingkan jumlah page fault
- Jawab:

Apakah page fault bertambah saat frame ditambah?

Jika iya → Terjadi Belady's Anomaly

3. Analisis Performa

Jawaban naratif (biasanya wajib di laporan):

- Algoritma terbaik:
 - ☞ Optimal (paling sedikit page fault, tapi tidak implementable)
- LRU vs FIFO:
 - ☞ LRU umumnya lebih baik karena mempertimbangkan locality
- FIFO lebih baik kapan?
 - ☞ Saat reference string sederhana & tidak menunjukkan locality **kuat**

4.4 Multi-level Page Table

1. Memory Overhead Comparison

Asumsi:

- Virtual address: 32-bit
- Page size: 4 KB
- Offset: 12 bit
- Page number: 20 bit
- Ukuran satu page table entry $\approx$ 8 byte
- Jumlah frame fisik (contoh): 1024 frame

Structure	Entries	Size per Entry	Total Memory	Space Efficiency
Single-Level	$2^{20} = 1.048.576$	8 byte	$\approx$ 8 MB	Rendah
Two-Level	Outer: 2^{10} Inner: on-demand	8 byte	$\sim$ puluhan KB – beberapa MB	Sedang
Three-Level	Multi-level on-demand	8 byte	Lebih kecil dari two-level	Tinggi
Inverted	= jumlah frame	8–12 byte	$\approx$ 8–12 KB	Sangat tinggi

Kesimpulan Memory Overhead:

Semakin banyak level page table, semakin efisien penggunaan memori untuk address space yang jarang digunakan. Inverted page table memiliki overhead paling kecil.

2. Translation Speed

Jumlah akses memori (tanpa TLB):

Struktur	Akses Page Table
----------	------------------

Struktur	Akses Page Table
Single-Level	1
Two-Level	2
Three-Level	3
Inverted	>1 (search / hashing)

Pengaruh TLB (Translation Lookaside Buffer)

- Jika TLB hit → translasi hanya 1 akses memori
- Jika TLB miss → akses page table sesuai level
- TLB sangat mengurangi overhead pada multi-level page table

Kesimpulan Kecepatan:

Single-level paling cepat tanpa TLB, tetapi dengan TLB, perbedaan kecepatan antar struktur menjadi tidak signifikan.

3. Kasus Penggunaan

Struktur mana yang cocok untuk sistem 32-bit vs 64-bit?

- 32-bit → Two-level atau Three-level
- 64-bit → Three-level atau lebih (bahkan 4–5 level)

Kapan inverted page table lebih efisien?

- Sistem dengan alamat virtual sangat besar
- Sistem dengan banyak proses
- Memori fisik terbatas
- Contoh: sistem modern dan server skala besar

4.5 Swapping Mechanism

1. Metrik Performa

Strategy Swap Out Count Swap In Count Total Swap Time Avg Response Time

FIFO	Tinggi	Tinggi	Tinggi	Lambat
LRU	Rendah	Rendah	Rendah	Cepat
Priority	Sedang	Sedang	Sedang	Sedang

Keterangan:

- Swap Out / Swap In Count dipengaruhi oleh seberapa sering proses harus dikeluarkan dan dimasukkan kembali ke memori.
- Total Swap Time meningkat seiring banyaknya operasi swapping.
- Average Response Time dipengaruhi oleh frekuensi swap dan pemilihan victim process.

2. Analisis

a. Strategi yang Meminimalkan Thrashing

Strategi LRU (Least Recently Used) paling efektif dalam meminimalkan thrashing karena memilih proses yang paling lama tidak diakses. Dengan mempertahankan proses yang sering digunakan di memori, LRU mengurangi frekuensi swap berulang yang menyebabkan thrashing.

b. Pengaruh Ukuran Memory terhadap Performa

Semakin besar ukuran memori fisik:

- Frekuensi swapping semakin rendah
- Response time sistem semakin cepat
- Thrashing lebih jarang terjadi

Sebaliknya, memori yang kecil akan menyebabkan sistem sering melakukan swap, sehingga performa menurun drastis.

c. Kapan Swapping Lebih Baik daripada Killing Process

Swapping lebih baik digunakan ketika:

- Proses masih dibutuhkan kembali dalam waktu dekat
- Sistem bersifat multi-tasking
- Proses memiliki state penting yang tidak boleh hilang

Killing process lebih cocok jika:

- Proses tidak responsif
- Prioritas sangat rendah
- Sistem berada dalam kondisi darurat memori

4.6

1. Perbandingan Spesifikasi Arsitektur

Aspek	Intel 32-bit (IA-32)	Intel 64-bit (x86-64)	ARMv8 (AArch64)
Arsitektur	CISC	CISC	RISC
Ukuran alamat virtual	32-bit	48-bit (umum)	48-bit (umum)
Maks. virtual memory	4 GB	256 TB	256 TB
Page size default	4 KB	4 KB	4 KB
Large page	4 MB	2 MB / 1 GB	2 MB / 1 GB
Multi-level paging	2-level	4–5 level	4 level
TLB	Lebih kecil	Lebih besar	Efisien & scalable
Penggunaan umum	Sistem lama	Desktop/Server modern	Mobile/Server

2. Penjelasan

a. Berapa bit address space yang digunakan?

- Intel 32-bit (IA-32): menggunakan 32-bit address space, sehingga alamat virtual maksimum adalah 2^{32} byte = 4 GB.
- Intel 64-bit (x86-64): secara teoritis 64-bit, tetapi implementasi umum menggunakan 48-bit virtual address, menghasilkan hingga 256 TB.
- ARMv8 (AArch64): juga mendukung 48-bit virtual address pada implementasi standar, dengan kapasitas 256 TB.

b. Bagaimana implementasi multi-level paging?

- IA-32: menggunakan two-level page table (Page Directory dan Page Table) untuk membagi alamat virtual.
- x86-64: menggunakan 4-level atau 5-level page table (PML4, PDPT, PD, PT) untuk mengelola address space yang sangat besar secara efisien.
- ARMv8: menggunakan 4-level translation table yang fleksibel dan mendukung berbagai ukuran page.

Multi-level paging mengurangi memory overhead karena page table hanya dialokasikan untuk bagian address space yang benar-benar digunakan.

c. Berapa ukuran maksimum virtual memory?

- Intel 32-bit: maksimal 4 GB virtual memory.
- Intel 64-bit: hingga 256 TB (bergantung implementasi OS dan CPU).
- ARMv8: hingga 256 TB, tergantung konfigurasi dan sistem operasi.

BAB V

Kesimpulan

Berdasarkan hasil praktikum dan analisis yang telah dilakukan, dapat disimpulkan bahwa arsitektur memori dan mekanisme manajemen memori sangat berpengaruh terhadap performa dan efisiensi sistem operasi. Sistem 64-bit (x86-64 dan ARMv8) memiliki keunggulan signifikan dibandingkan 32-bit (IA-32) karena mendukung address space yang jauh lebih besar, sehingga mampu menangani aplikasi modern dengan kebutuhan memori tinggi.

Implementasi multi-level page table terbukti efektif dalam mengurangi overhead memori dengan hanya mengalokasikan page table untuk bagian address space yang digunakan. Selain itu, keberadaan TLB dan hierarki cache (L1, L2, L3) berperan penting dalam mempercepat translasi alamat dan akses memori, sehingga meningkatkan kinerja sistem secara keseluruhan.

Dari simulasi mekanisme swapping, strategi LRU cenderung memberikan performa terbaik dalam meminimalkan thrashing karena mempertimbangkan pola akses proses, sementara FIFO lebih sederhana namun kurang efisien. Strategi priority-based efektif pada sistem yang memerlukan jaminan layanan untuk proses penting.

Secara keseluruhan, pemilihan arsitektur, ukuran memori, serta strategi manajemen memori dan swapping harus disesuaikan dengan kebutuhan sistem untuk mencapai keseimbangan antara performa, efisiensi, dan stabilitas sistem.